
MP3: Coordinate Spaces

Due Time: Refer to the course web-site

Objectives

Verify that we understand and are able to work with:

- Unity specific skills:
 - System-wide shared Material and GameObject specific instances
 - Cameras: editor and main game window cameras
 - Multiple cameras and viewports
 - Unity defined shader variables: `_SinTime` or `_CosTime`
- Shader and relevant general concepts:
 - Coordinate transformation pipeline: Object, World, View (Eye, or Camera), and Perspective
 - Corresponding positions in different coordinate spaces

Approach

To examine and verify different coordinate spaces along the transformation pipeline, we will create the simple “*vanishing effect*” in different coordinate spaces. The vanishing effect is where an object will shrink towards and disappears at a user specified position. A straightforward implementation of this effect is by *moving* all vertex positions, p , towards the *target*, or the vanishing point:

$$p.xyz += w * (target - p)$$

In the above expression, when the weight, w , is 0.0, $(target - p)$ is scaled to zero, and thus the vertex position is not changed. When w becomes 1.0, the p on left- and right-side of the expression cancels off and the resulting position is at the *target*. In our case, p represents all vertex positions, as all vertices converges towards the *target*, a *vanishing effect* towards *target* position can be observed when the weight, w , changes from 0.0 to 1.0.

In this assignment, we will use this straightforward mechanism to examine and work with positions in different coordinate space: Object Coordinate (OC), World Coordinate (WC), Eye Coordinate (EC), and Perspective Coordinate (PC) spaces. You will develop a five (5) pass shader. Each pass is to implement the vanishing functionality in each of the 4 coordinate spaces (OC, WC, EC, and PC), where the last pass is to render the original geometry. Yes, this is rather inefficient, but, this is an exercise.

In each pass, you must support the following user controls:

- Weight: the w parameter in the above equation.
- Show/Hide: of the render results. I use the *discard* command in the fragment shader.
- Animate: ignore the weight and control w via the Unity shader variables `_SinTime` or `_CosTime`
- VPoint: allow user specify a vanishing position in the corresponding space

These control parameters are slightly different for each of the spaces. Details are as follow:

Object Coordinate (OC) space: by default, this pass vanishes geometries towards the origin of OC.

- Parameters:
 - OC_Weight: the w parameter controlling the vanishing in OC space
 - OC_Show: show or hide the rendered results of this pass
 - OC_Animate: when true, ignore OC_Weight and animate the weight according to:
$$w = 0.5 * (1 + \sinTime.z)$$
 - OC_VPoint: a position in OC space
 - OC_Use_VPoint: when true use the user adjustable OC_VPoint as the *target*.
- Here is [an example for reference](#) (OC_Animate is set to true)
 - Time 0:00 to 0:15: Teapot vanishing towards the origin of OC
 - Time 0:16 to 0:23: Vanishing towards the OC_VPoint
 - Time 0:23 to the end: OC_VPoint interactively adjusted with y-value increased

World Coordinate (WC) space: this pass vanishes geometries towards the position of a user specified GameObject

- Parameters:
 - WC_Weight: the w parameter controlling the vanishing in WC space (note: unless otherwise explained, the vanishing operations in different coordinate spaces are completely independent).
 - WC_Show: show or hide the rendered results of this pass
 - WC_Animate: when true, ignore WC_Weight and animate the weight according to:
$$w = \text{pow}(\text{abs}(_SinTime.z), \text{WC_Rate})$$
 - WC_Rate: a floating point value for scaling the vanishing oscillation. The point of this variation is for us to appreciate the shape and behavior of sinusoidal function when raised to a power. Try raising WC_Rate to a very large number and notice how fast the vanishing operation becomes.
 - WC_VPoint: a reference to a GameObject, where the current object location will be used as the target position.
 - Note: this is reference to a GameObject such that at runtime, you can move an object to change the vanishing position.
 - WC_Use_OC_VPoint: transform the OC_VPoint to WC and use as the target for vanishing.
Watch out, in OC space, the Teapot's "up" is Z
- Here is [an example for reference](#) (WC_Animate is set to true)
 - Time 0:00 to 0:18: Teapot vanishing towards the position of the barrel
 - Time 0:19 to the end: WC_Use_OC_VPoint is set to true and vanishing towards the OC_VPoint in WC space. This means, if you manipulate the position/rotation/scale of the teapot, the relative position of the vanishing point (at the bottom of the teapot) will not change, i.e., the vanishing position will be transformed along with the teapot. Here is an [example showing this result](#).
 - Note:** when vanishing towards OC_VPoint in WC, the vanishing process is identical to that of when computed in OC (that's why we have different vanishing rate). The point here is, the same computation can be accomplished in difference coordinate spaces. Here is an [example showing both OC \(red\) and WC \(blue\) computations](#) of vanishing toward OC_VPoint. Note the red is computed in OC-pass and blue is computed in WC-pass.

Eye Coordinate (EC) space pass: vanishes towards the far plane at the center of the camera

- EC is also referred to as the View or Camera Coordinate space
- Parameters:
 - EC_Weight: the w parameter controlling the vanishing in EC space
 - Once again, the EC vanishing control is independent from that of OC and WC
 - Remember EC space, where the eye position is located at the origin looking down the negative Z-axis. The far clipping plane, f , is located at $(0, 0, -f)$.
 - When object vanishes toward the far plane location, they becomes smaller while moving toward the middle of the game window.
 - You can verify the vanishing process by placing an object *in front* occluding the vanishing object. During the vanishing process, you will observe the vanishing object growing smaller and move from behind to in front of the occluding object.
 - WATCH OUT:** remember that by default the far clipping plan is located at 1000 units away. A w of 0.5 means, the teapot will be moved to 500 units away from the camera and will appear very small. In this case, the "active" or usable range of w is with very small values (e.g., between the range of 0.0x. You can fix this by decreasing the far clipping plane distance, e.g., to 30. Try it.
 - EC_Animate: when true, ignore EC_Weight and animate the weight according to:
$$w = 0.5 * (1 + _CosTime.z)$$
 - EC_Use_OC_VPoint: transform the OC_VPoint to EC and use as the target for vanishing.
 - Note: in this case the vanishing point will be exactly the same as the OC_VPoint in the OC vanishing process.
 - EC_Use_WC_VPoint: transform the WC_VPoint to EC and use as the target for vanishing.
 - Note: in this case the vanishing point will be exactly the same as the WC_VPoint in the WC vanishing process.

- Note: in this case the vanishing point will be exactly the same as the OC_VPoint in the previous cases.
 - PC_Use_WC_VPoint: transform the WC_VPoint to PC and use as the target for vanishing.
 - Note: in this case the vanishing point will be exactly the same as the WC_VPoint in the previous cases.
 - If both PC_Use_OC_VPoint and PC_Use_WC_VPoint are enabled at the same time, you are free to choose which to use.
- Here is [an example for reference](#) (PC_Animate is set to true)
 - Note that since PC is specific to a display and in this case the PC corresponds to the game window, the effects of PC vanishing should be observed in the game window (and not the editor window).
 - Time 0:00 to 0:15: Teapot vanishing towards PC_VPoint at (0.5, 0.5) or, center point of bottom-right quadrant (remember y-direction is flipped).
 - Time 0:16 to 0:27: PC_VPoint is changed to (0.5, -0.5), and we can observe the teapot vanishing towards the center of the top-right quadrant.
 - Time 0:16 to the end: PC_Use_WC_VPoint is switched on, and vanishing towards the barrel in identical fashion as the WC operation.

Lastly, here is [an example showing all passes working](#) in the same rendering. In this example, except for the rates, the vanishing operations are identical. Once again, in general we are free to choose to perform the operation in the most convenient space. One important lesson learned here is that, ***all geometries (e.g., vertex position, vanishing position) involved in a computation must be in the same coordinate space!***

Unity scene setup: Your Unity scene must be defined to support the convenient testing of the above functionality. Here are the details:

- You will implement a shader supporting the above functionality and associate the shader with a Unity material.
 - Object color: your material must also allow the changing of object color such individual objects can be readily differentiated.
 - Show/hide the original object: you must allow the show/hide of the rendering of the original object
 - Rendered object color from each pass must be unique:
 - OC: (1, 0, 0, 1)
 - WC: (0, 0, 1, 1);
 - EC: (0.5, 1, 1, 1);
 - PC: (0.5, 0.2, 0.2, 1);
 - Original: User defined
- This material will be assigned to at least 5 objects where:
 - Material instancing: At least two of the objects must modify their materials (material instancing), these two objects must be with unique colors.
 - Material sharing: At least three of the objects must shared the system-wide material (where the modifications of the material are performed on the material object in the Asset folder).

Hint: Once again, the number of lines of code is relatively small but you need to know what you are doing. Here are some details to watch out for.

- Here are the potential templates for implementation, feel free to ignore these, but you are also very welcome to base your implementation on these:
 - [MaterialControl.cs](#) definitions (including the bit-mask flags)
 - [Structs.cginc](#): GPU-side bitmask definition
 - [552_MP3_Shader.shader](#): frame for my shader implementation

In all cases, please note that if you choose to use the given source code, then, it is ***yours***. You are responsible to study and understand every single line of the code. You are free to modify any

- Given OC experience from MP2, you may want to implement the OC pass and make sure everything is working as expected. In my implementation, the OC pass is less than 50-lines of code (including declarations and blank lines).

- For organization and ease of debugging, I separated all the passes into different cginc files.
- To support proper capturing of translation, *positions must be of type float4 in all transformations*, matrix to position multiplications, you can conveniently convert a float3 to float4 by: float4(p, 1) [where p is a float3]
 - **I define all my shader vanishing points as float3, and do float4(p, 1) for all mul() operations!**
 - Make sure you understand this point, otherwise, all your V-points may be stuck at the world origin!

Credit Distribution

	Credits
1 Unity Scene Setting	
Material: instancing (2) and Sharing (3)	5
Show/Hide and unique color for each pass	5
Able to modify all parameters	5
2 OC Pass	
Show/Hide	2
Interactive control of w	5
Proper animation	4
Use Vpoint support	6
3 WC Pass	
Show/Hide	2
Interactive control of w and rate	2
Proper animation	2
Use Vpoint support	5
Use OC_Vpoint Support	6
4 EC Pass	
Show/Hide	2
Interactive control of w	2
Proper animation	6
Use OC and WC Vpoint support	8
Only_Z support	2
5 PC Pass	
Show/Hide	2
Interactive control of w	2
Proper animation	2
Use VPoint support	5
Use OC and WC Vpoint support	10
6 Submission	
Proper project name	5

Four folders in submission	5
Total	100

This programming assignment counts 10% towards your final grade for this class.